

Day 17: MLflow for Machine Learning Lifecycle Management

5-line beginner summary

1. **MLflow is a tool to manage the full ML lifecycle:** experiments, models, versions, deployment, and monitoring.
 2. It helps you remember **which model was trained with which data, parameters, code, and results.**
 3. In MLflow, you log **parameters, metrics, artifacts, and models** inside an experiment run.
 4. The **Model Registry** helps teams version, approve, promote, deploy, and roll back models.
 5. For GenAI, MLflow is now also used for **LLM tracing, evaluation, prompt/app versioning, and observability.** ([Databricks Documentation](#))
-

1. What MLflow is

MLflow is an open-source AI/ML lifecycle platform.

It helps data scientists and ML engineers manage:

- Experiment tracking
- Model packaging
- Model registry
- Model versioning
- Model deployment
- Model evaluation
- GenAI/LLM tracing and evaluation

In simple words:

| MLflow is like a **project diary + model library + deployment manager** for machine learning projects.

Example:

You train 10 fraud detection models. Without MLflow, you may forget:

- Which dataset was used?
- Which algorithm was used?
- What hyperparameters were used?
- Which model gave the best F1-score?
- Which version is running in production?

MLflow stores all this in one place.

MLflow documentation currently separates capabilities for **traditional ML workflows** and **LLM/agent workflows**, including tracking, registry, deployment, tracing, and evaluation. ([MLflow AI Platform](#))

2. Why experiment tracking matters

In machine learning, we rarely train only one model.

We try many combinations:

- Logistic Regression
- Random Forest
- XGBoost
- Different learning rates
- Different train/test splits
- Different feature sets
- Different preprocessing logic

Without experiment tracking, your work becomes messy.

Without MLflow

```
model_final.pkl  
model_final_new.pkl  
model_best.pkl  
model_best_latest.pkl  
model_v2_really_final.pkl
```

This is dangerous.

You may not know which model is actually best.

With MLflow

Each training attempt becomes a **run**.

For each run, MLflow stores:

```
Run ID  
Algorithm  
Parameters  
Metrics  
Artifacts  
Model file  
Code version  
Dataset info  
Timestamp  
User
```

Databricks documentation explains that MLflow experiments organize runs so teams can log parameters, metrics, artifacts, and code versions while comparing model performance. ([Databricks Documentation](#))

3. Parameters

Parameters are input settings used during training.

They are usually chosen before model training.

Examples:

```
model_type = RandomForest
n_estimators = 200
max_depth = 8
learning_rate = 0.05
train_test_split = 80/20
embedding_model = sentence-transformers/all-MiniLM
chunk_size = 500
```

Simple meaning:

| Parameters answer: "What settings did I use?"

Easy example

For a customer churn model:

```
Algorithm: RandomForest
n_estimators: 100
max_depth: 5
class_weight: balanced
```

If the model performs well, MLflow helps you remember exactly how it was trained.

4. Metrics

Metrics are output results used to judge model performance.

Examples:

For classification:

```
accuracy
precision
recall
F1-score
ROC-AUC
confusion matrix
```

For regression:

```
MAE
MSE
RMSE
R2 score
```

For RAG/GenAI:

```
answer relevance
faithfulness
groundedness
context precision
context recall
toxicity
```

```
latency
cost
human feedback score
```

Simple meaning:

| Metrics answer: “How good was the model?”

Example:

```
Run 1:
accuracy = 0.88
f1_score = 0.81

Run 2:
accuracy = 0.91
f1_score = 0.86
```

Now you can compare and choose Run 2.

5. Artifacts

Artifacts are files produced during training or evaluation.

Examples:

```
trained_model.pkl
feature_importance.png
confusion_matrix.png
classification_report.txt
preprocessing_pipeline.pkl
training_dataset_snapshot.csv
evaluation_results.json
prompt_template.txt
RAG evaluation report
```

Simple meaning:

| Artifacts answer: “What files were created during the experiment?”

For example, if you generate a confusion matrix image, MLflow can store it with the run.

6. Model Registry

The **MLflow Model Registry** is a central place to manage trained models.

It helps teams manage:

- Registered models
- Model versions
- Model aliases
- Tags
- Metadata
- Lineage

- Approval status
- Deployment readiness

MLflow's official documentation describes the Model Registry as a centralized model store with APIs and UI for managing the model lifecycle, including lineage, versioning, aliases, metadata tags, and annotations. ([MLflow AI Platform](#))

Simple example

You create a registered model:

```
CustomerChurnModel
```

Inside it, you may have:

```
Version 1: Logistic Regression
Version 2: Random Forest
Version 3: XGBoost
Version 4: XGBoost with better features
```

The registry helps answer:

```
Which model is latest?
Which model is approved?
Which model is in production?
Which model should we roll back to?
Who trained it?
Which experiment created it?
```

7. Model versioning

Every time you register a new model under the same model name, MLflow creates a new version.

Example:

```
Registered Model: LoanDefaultModel

Version 1:
- Algorithm: Logistic Regression
- F1-score: 0.72

Version 2:
- Algorithm: Random Forest
- F1-score: 0.79

Version 3:
- Algorithm: XGBoost
- F1-score: 0.83
```

MLflow Registry concepts include **Registered Model**, **Model Version**, **Model URI**, and **Model Alias**. A model can be addressed by version, such as `models:/MyModel/1`, or by alias, such as `models:/MyModel@champion`. ([MLflow AI Platform](#))

Important modern concept: aliases

Instead of hardcoding version numbers in production code, use aliases:

```
@champion -> current best production model
@challenger -> new model being tested
@baseline -> safe older model
```

Example:

```
models:/LoanDefaultModel@champion
```

This means:

| Load whichever version is currently marked as champion.

If version 3 is bad, you can move @champion back to version 2.

That is rollback.

8. Model promotion from development to production

Model promotion means moving a model through controlled lifecycle steps.

Typical flow:

```
Development -> Validation -> Staging -> Production
```

Or in newer alias-based style:

```
candidate -> challenger -> champion
```

Example promotion flow

1. Data scientist trains model.
2. MLflow logs experiment.
3. Best model is registered.
4. Validation tests are run.
5. Model gets approval.
6. Alias @challenger is assigned.
7. Shadow testing or A/B testing is done.
8. If good, alias @champion points to new model.
9. Serving endpoint uses @champion.

Databricks-managed MLflow integrates tracking, registry, Unity Catalog governance, and Model Serving, where serving deploys models to REST API endpoints. ([Databricks Documentation](#))

9. Reproducibility

Reproducibility means you can recreate the same result later.

A good MLflow run should store:

```
Code version
Dataset version
Feature logic
Parameters
Metrics
Artifacts
Model file
Library dependencies
Environment details
Random seed
Training time
Evaluation result
```

Simple example:

Imagine an interviewer asks:

| “How do you know this production model was trained correctly?”

Good answer:

| “Because every model version is linked to an MLflow run. That run contains parameters, metrics, artifacts, model files, dependencies, and lineage. So we can trace how the model was built and reproduce or audit it.”

The Model Registry documentation says model lineage links a registered model version back to the MLflow run, logged model, or notebook that produced it, helping with traceability and reproducibility. ([MLflow AI Platform](#))

10. Model deployment basics

Deployment means making the model available for prediction.

Common deployment patterns:

```
Batch inference
Real-time REST API
Streaming inference
Embedded model inside application
Scheduled job
Databricks Model Serving endpoint
Kubernetes endpoint
Cloud endpoint
```

Simple REST API example

Input:

```
{
  "age": 42,
  "income": 80000,
  "loan_amount": 300000
}
```

Output:

```
{  
  "default_probability": 0.18,  
  "risk_category": "low"  
}
```

MLflow Serving helps deploy models to local environments, cloud services, and Kubernetes. MLflow packages the model with dependencies and can launch an inference server with REST endpoints. ([MLflow AI Platform](#))

11. MLflow with Databricks

Databricks uses MLflow as a core MLOps tool.

In Databricks, MLflow can connect with:

```
Databricks notebooks  
Experiments  
Jobs  
Feature Store  
Unity Catalog  
Model Registry  
Model Serving  
Lakehouse data  
Delta Lake  
GenAI apps and agents
```

Typical Databricks ML lifecycle:

```
Delta Lake data  
  ↓  
Feature engineering  
  ↓  
Train model in notebook/job  
  ↓  
Track experiment with MLflow  
  ↓  
Register model in Unity Catalog  
  ↓  
Promote model version  
  ↓  
Deploy with Model Serving  
  ↓  
Monitor performance
```

Databricks documentation says MLflow on Databricks supports developing generative AI agents and ML models, and it uses Unity Catalog and the cloud data lake to unify data and AI assets across the ML lifecycle. ([Databricks Documentation](#))

12. MLflow for GenAI evaluation

MLflow is no longer only for traditional ML models.

It is also used for GenAI applications such as:

```
RAG applications
Chatbots
Agents
Prompt chains
LangChain apps
LangGraph workflows
Tool-calling agents
LLM evaluation pipelines
```

For GenAI, MLflow can track:

```
Prompt version
Input question
Retrieved context
LLM response
Tool calls
Intermediate steps
Latency
Token usage
Cost
Human feedback
Evaluation scores
Groundedness
Faithfulness
Relevance
```

MLflow 3 for GenAI unifies tracking, evaluation, and observability for GenAI apps and agents. It includes real-time trace logging, built-in and custom scorers, human feedback, and version tracking. ([Databricks Documentation](#))

Simple RAG example

Question:

```
What is the company leave policy?
```

MLflow can log:

```
User question
Retrieved policy chunks
Prompt sent to LLM
Generated answer
Source citations
Faithfulness score
Relevance score
Latency
Human feedback
```

This is very useful in enterprise GenAI because you need to prove:

```
The answer came from approved documents.
The response was grounded.
```

The prompt version is known.
The app behavior is traceable.

13. Model lineage

Model lineage means knowing the full history of a model.

It answers:

Which data trained this model?
Which notebook or job trained it?
Which code version was used?
Which parameters were used?
Which metrics were achieved?
Who approved it?
Which version went to production?
Which endpoint is using it?

Simple meaning:

| Lineage is the family tree of a model.

Example:

```
Production Model: CustomerChurnModel v7
  ↓ created from
MLflow Run: run_9821
  ↓ used
Notebook: train_churn_model.py
  ↓ used
Dataset: customer_features_delta_table version 43
  ↓ used
Parameters: max_depth=8, learning_rate=0.05
  ↓ produced
Metrics: f1_score=0.87, roc_auc=0.91
```

This is important for enterprise governance, audit, compliance, debugging, and rollback.

14. Rollback strategy

Rollback means returning to a previous safe model when the current model has problems.

Reasons for rollback:

Prediction quality drops
Latency increases
Data drift occurs
Model gives biased output
API errors increase
Business users complain
GenAI app starts hallucinating
Cost becomes too high

Good rollback strategy

1. Never overwrite production model manually.
2. Always deploy using model version or alias.
3. Keep previous stable version available.
4. Monitor production metrics.
5. If issue occurs, move @champion alias back to older version.
6. Restart or refresh serving endpoint if needed.
7. Log rollback reason.
8. Investigate failed model offline.

Example:

```
Before rollback:
LoanDefaultModel@champion -> version 5

Issue:
Version 5 has high false negatives.

Rollback:
LoanDefaultModel@champion -> version 4
```

Because aliases are mutable references to model versions, they make controlled promotion and rollback easier. ([MLflow AI Platform](#))

Easy example: Customer churn model

Imagine a telecom company wants to predict whether a customer will leave.

Without MLflow

A data scientist trains many models:

```
Logistic Regression
Random Forest
XGBoost
LightGBM
```

But results are saved manually in Excel.

Problems:

```
Hard to compare
Hard to reproduce
Hard to deploy
Hard to audit
Hard to roll back
```

With MLflow

Each training run is logged:

```
Experiment: customer_churn_prediction
```

```
Run 1:
model = LogisticRegression
accuracy = 0.82
f1_score = 0.76
```

```
Run 2:
model = RandomForest
accuracy = 0.86
f1_score = 0.81
```

```
Run 3:
model = XGBoost
accuracy = 0.89
f1_score = 0.84
```

Best model is registered:

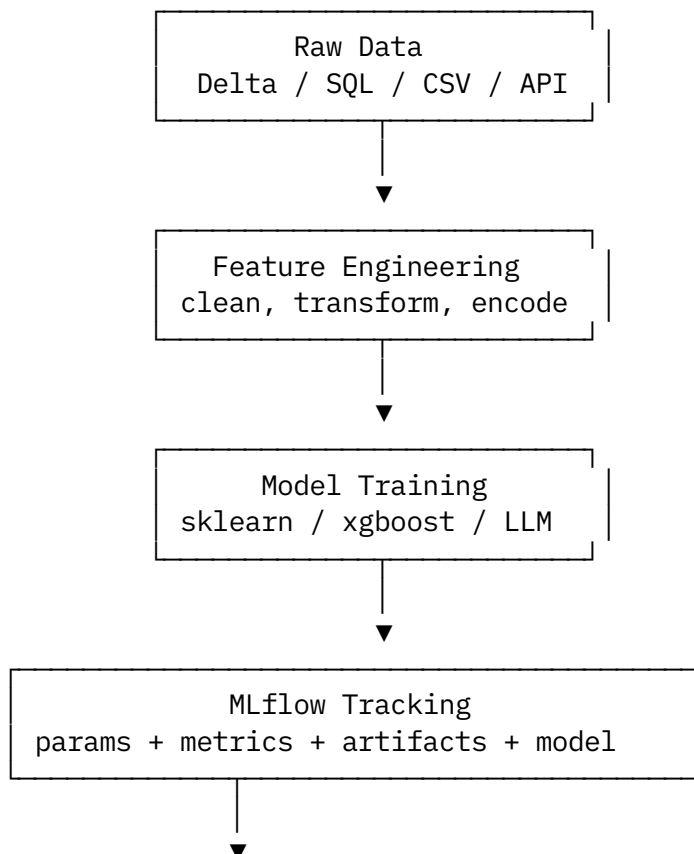
```
Registered Model: CustomerChurnModel
Version: 3
Alias: @champion
```

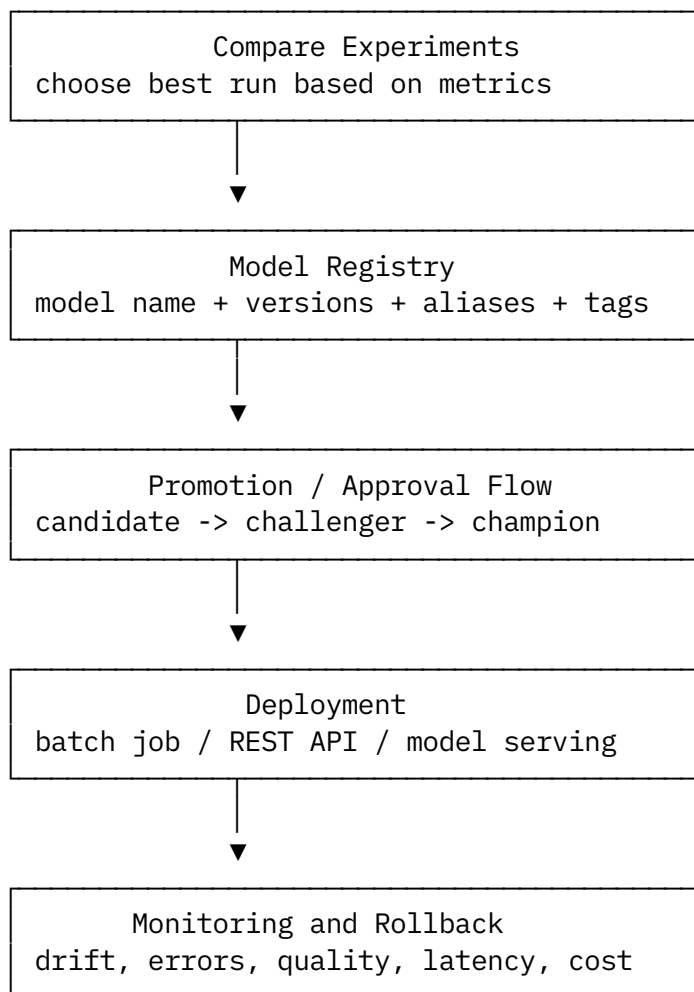
Production API uses:

```
models:/CustomerChurnModel@champion
```

If version 3 fails, rollback to version 2.

ASCII diagram: MLflow lifecycle





Pseudocode for tracking an experiment

```
# Pseudocode: MLflow experiment tracking

import mlflow
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

# 1. Set experiment name
mlflow.set_experiment("customer_churn_prediction")

# 2. Start one MLflow run
with mlflow.start_run(run_name="random_forest_baseline"):

    # 3. Define parameters
    n_estimators = 100
    max_depth = 8
    random_state = 42

    # 4. Log parameters
    mlflow.log_param("model_type", "RandomForestClassifier")
    mlflow.log_param("n_estimators", n_estimators)
    mlflow.log_param("max_depth", max_depth)
```

```

mlflow.log_param("random_state", random_state)

# 5. Train model
model = RandomForestClassifier(
    n_estimators=n_estimators,
    max_depth=max_depth,
    random_state=random_state
)

model.fit(X_train, y_train)

# 6. Make predictions
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# 7. Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_prob)

# 8. Log metrics
mlflow.log_metric("accuracy", accuracy)
mlflow.log_metric("f1_score", f1)
mlflow.log_metric("roc_auc", auc)

# 9. Log artifacts
mlflow.log_artifact("confusion_matrix.png")
mlflow.log_artifact("classification_report.txt")

# 10. Log trained model
mlflow.sklearn.log_model(
    sk_model=model,
    artifact_path="model"
)

# 11. Add useful tags
mlflow.set_tag("project", "customer_churn")
mlflow.set_tag("environment", "development")
mlflow.set_tag("owner", "data_science_team")

```

Pseudocode for registering and deploying a model

```

# Pseudocode: Register best model and prepare for deployment

import mlflow
from mlflow.tracking import MlflowClient

client = MlflowClient()

# 1. Find the best run from experiment
experiment_name = "customer_churn_prediction"
experiment = mlflow.get_experiment_by_name(experiment_name)

```

```

runs = mlflow.search_runs(
    experiment_ids=[experiment.experiment_id],
    order_by=["metrics.f1_score DESC"],
    max_results=1
)

best_run_id = runs.iloc[0]["run_id"]
best_f1 = runs.iloc[0]["metrics.f1_score"]

# 2. Build model URI from best run
model_uri = f"runs:{best_run_id}/model"

# 3. Register model
registered_model_name = "CustomerChurnModel"

model_version = mlflow.register_model(
    model_uri=model_uri,
    name=registered_model_name
)

# 4. Add tags to the model version
client.set_model_version_tag(
    name=registered_model_name,
    version=model_version.version,
    key="validation_status",
    value="pending"
)

client.set_model_version_tag(
    name=registered_model_name,
    version=model_version.version,
    key="f1_score",
    value=str(best_f1)
)

# 5. After validation checks pass, promote using alias
client.set_registered_model_alias(
    name=registered_model_name,
    alias="challenger",
    version=model_version.version
)

# 6. Run shadow testing or business validation
# Example:
# compare challenger predictions with champion predictions

validation_passed = True

if validation_passed:
    # 7. Promote challenger to champion
    client.set_registered_model_alias(
        name=registered_model_name,
        alias="champion",

```

```

        version=model_version.version
    )

# 8. Deployment system loads champion model
production_model_uri = "models:/CustomerChurnModel@champion"

model = mlflow.pyfunc.load_model(production_model_uri)

# 9. Serve model using REST endpoint / batch job / Databricks Model Serving
predictions = model.predict(new_customer_data)

```

Pseudocode for rollback

```

# Pseudocode: Rollback production model to previous stable version

from mlflow.tracking import MlflowClient

client = MlflowClient()

registered_model_name = "CustomerChurnModel"

# Current production version has issue
bad_version = 5

# Previous stable version
stable_version = 4

# Move champion alias back to stable version
client.set_registered_model_alias(
    name=registered_model_name,
    alias="champion",
    version=stable_version
)

# Add tags for audit
client.set_model_version_tag(
    name=registered_model_name,
    version=bad_version,
    key="rollback_status",
    value="rolled_back_due_to_quality_drop"
)

client.set_model_version_tag(
    name=registered_model_name,
    version=stable_version,
    key="rollback_reason",
    value="restored_after_version_5_quality_issue"
)

```


Pseudocode for GenAI evaluation with MLflow

```
# Pseudocode: Track and evaluate a RAG application

import mlflow

mlflow.set_experiment("enterprise_policy_rag_eval")

with mlflow.start_run(run_name="rag_prompt_v3"):

    # 1. Log RAG configuration
    mlflow.log_param("embedding_model", "text-embedding-model")
    mlflow.log_param("vector_db", "FAISS_or_Databricks_Vector_Search")
    mlflow.log_param("top_k", 5)
    mlflow.log_param("chunk_size", 500)
    mlflow.log_param("prompt_version", "v3")

    # 2. Run evaluation dataset
    for question in golden_questions:

        retrieved_chunks = retriever.search(question, top_k=5)

        answer = llm.generate(
            question=question,
            context=retrieved_chunks,
            prompt_template=prompt_v3
        )

        # 3. Calculate evaluation scores
        relevance_score = evaluate_relevance(question, answer)
        groundedness_score = evaluate_groundedness(answer, retrieved_chunks)
        faithfulness_score = evaluate_faithfulness(answer, retrieved_chunks)

        # 4. Log metrics
        mlflow.log_metric("avg_relevance", relevance_score)
        mlflow.log_metric("avg_groundedness", groundedness_score)
        mlflow.log_metric("avg_faithfulness", faithfulness_score)

    # 5. Log artifacts
    mlflow.log_artifact("rag_eval_report.json")
    mlflow.log_artifact("prompt_template_v3.txt")
```

Common mistakes

1. Logging only accuracy

Accuracy alone is not enough.

For imbalanced data, use:

```
precision
recall
F1-score
```

```
ROC-AUC
confusion matrix
```

For GenAI, use:

```
groundedness
faithfulness
relevance
latency
cost
human feedback
```

2. Not logging parameters

Bad practice:

```
mlflow.log_metric("accuracy", 0.91)
```

Better:

```
mlflow.log_param("model_type", "XGBoost")
mlflow.log_param("max_depth", 6)
mlflow.log_param("learning_rate", 0.05)
mlflow.log_metric("accuracy", 0.91)
```

Without parameters, you know the result but not how you got it.

3. Not logging artifacts

Do not log only numbers.

Also log:

```
confusion matrix
feature importance
evaluation report
preprocessing pipeline
prompt template
requirements file
```

4. Registering every bad model

Do not register every experiment model.

Better flow:

```
Track all runs
Compare metrics
Select best candidate
Validate properly
Register only useful models
Promote only approved models
```

5. Confusing experiment tracking and model registry

They are related but different.

Concept	Purpose
Experiment Tracking	Compare training runs
Model Registry	Manage approved model versions
Model Serving	Deploy model for prediction
Monitoring	Check production behavior

6. No rollback plan

Never deploy a model without knowing how to go back.

Bad:

```
Overwrite production model file manually.
```

Good:

```
Use registry versioning and aliases.  
Move @champion back to previous stable version if needed.
```

7. Not tracking data version

A model is not reproducible if you do not know which data trained it.

Track:

```
training table  
Delta table version  
feature table version  
data extraction date  
data filters  
train/test split logic
```

8. Ignoring environment dependencies

A model may work in a notebook but fail in production because library versions differ.

Track:

```
Python version  
scikit-learn version  
xgboost version  
transformers version  
requirements.txt  
conda.yaml  
Docker image
```

MLflow deployment documentation highlights that MLflow packages models with dependencies and environment information so the model can run consistently across environments. ([MLflow AI Platform](#))

9. Treating GenAI evaluation like normal classification

GenAI answers are not always simply right or wrong.

You need to evaluate:

```
correctness
groundedness
faithfulness
context quality
toxicity
latency
cost
human feedback
```

MLflow 3 for GenAI supports tracing, built-in/custom scorers, LLM judges, human feedback, evaluation, monitoring, and app/prompt versioning. ([Databricks Documentation](#))

Interview-ready explanation

You can say:

MLflow is used to manage the machine learning lifecycle. During training, I use MLflow Tracking to log parameters, metrics, artifacts, model files, and code or data references. After comparing runs, I register the best model in the Model Registry, where it gets a version, metadata, lineage, and aliases such as challenger or champion. In production, the serving layer loads the approved model alias instead of hardcoding a version. If the new model performs badly, I can roll back by moving the champion alias to the previous stable version. On Databricks, MLflow integrates with notebooks, jobs, Unity Catalog, Feature Store, Model Registry, and Model Serving. For GenAI, MLflow can also track prompts, traces, retrieved context, LLM responses, evaluation scores, human feedback, and app versions.

One simple mental model

```
MLflow Tracking  = What did I try?
MLflow Metrics   = How good was it?
MLflow Artifacts = What files were produced?
MLflow Registry  = Which model version is approved?
MLflow Serving   = How is the model used by applications?
MLflow Lineage   = Where did this model come from?
MLflow Rollback  = How do I return to a safe version?
```

For IBM AI/GenAI interviews, focus on this:

MLflow is not just a tool for saving models. It is a lifecycle management system that helps teams track experiments, reproduce results, govern model versions, deploy safely, monitor quality, and roll back when needed.